information disclosure threats. Imagine that you were trying to check the latest gossip from your work party while connected to a public wifi hotspot in your local coffee shop. Without encryption, the messages you send to and from the API will be readable by anybody else connected to the same hotspot.

Your simple password authentication scheme is also vulnerable to this snooping, as an attacker with access to the network can simply read your Base64-encoded passwords as they go by. They can then impersonate any user whose password they have stolen. It’s often the case that threats are linked together in this way. An attacker can take advantage of one threat, in this case information disclosure from unencrypted communications, and exploit that to pretend to be somebody else, undermining your API’s authentication. Many successful real-world attacks result from chaining together multiple vulnerabilities rather than exploiting just one mistake.

In this case, sending passwords in clear text is a pretty big vulnerability, so let’s fix that by enabling HTTPS. HTTPS is normal HTTP, but the connection occurs over Transport Layer Security (TLS), which provides encryption and integrity protection. Once correctly configured, TLS is largely transparent to the API as it occurs at a lower level in the protocol stack and the API still sees normal requests and responses. Figure 3.5 shows how HTTPS fits into the picture, protecting the connections between our users and the API.

In addition to protecting data in transit (on the way to and from our application), you should also consider protecting